

# BFC Deposit Assistant — Developer Manual

---

A domain-specific RAG (Retrieval-Augmented Generation) chatbot that answers questions about Best Finance Company (BFC) deposit products. Responses are grounded strictly in source documents — no hallucination.

---

## Table of Contents

- 1. [Project Structure](#)
  - 2. [Tech Stack](#)
  - 3. [Setup & Running](#)
  - 4. [Architecture Overview](#)
  - 5. [Document Loading & Parsing](#)
  - 6. [Embeddings](#)
  - 7. [Vector Store \(Milvus\)](#)
  - 8. [Ingestion Pipeline](#)
  - 9. [LLM & Prompts](#)
  - 10. [RAG Chain](#)
  - 11. [Multi-Turn Conversation](#)
  - 12. [Configuration Reference](#)
  - 13. [Infrastructure \(Docker\)](#)
  - 14. [Adding New Documents](#)
  - 15. [Common Issues](#)
- 

## 1. Project Structure

```
rag_best/
├── rag/                                # Core application package
│   ├── __init__.py
│   ├── config.py                      # All config & env variables
│   ├── loader.py                      # DOCX parser → LangChain documents
│   ├── vectorstore.py                 # Embeddings + Milvus integration
│   └── chain.py                       # Prompts + RAG chain with history
├── data/
│   ├── AI CHATBOT DEPOSITS.docx      # Primary source document
│   └── data.xlsx                     # Supplementary data
├── main.py                           # Entry point: run the chatbot
├── ingest.py                         # Entry point: load documents into Milvus
├── requirements.txt
├── docker-compose.yml                # Milvus + supporting services
└── .env                             # API keys (never commit this)
```

**Rule of thumb:**

- rag/ — library code, never run directly
- main.py / ingest.py — scripts you actually execute

## 2. Tech Stack

| Layer            | Technology                                                     | Version                   |
|------------------|----------------------------------------------------------------|---------------------------|
| LLM              | Groq (Meta Llama 4 Scout 17B)                                  | via API                   |
| LLM Framework    | LangChain Core                                                 | latest                    |
| Vector DB        | Milvus Standalone                                              | 2.4.3                     |
| Embeddings       | HuggingFace <code>paraphrase-multilingual-MiniLM-L12-v2</code> | via sentence-transformers |
| Document Parsing | python-docx                                                    | latest                    |
| Infrastructure   | Docker Compose                                                 | —                         |
| Vector DB UI     | Attu                                                           | latest                    |

## 3. Setup & Running

### Prerequisites

- Python 3.11+
- Docker Desktop running

### First-time setup

```
# 1. Clone and enter project
cd rag_best

# 2. Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate           # macOS/Linux
# venv\Scripts\activate           # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Set up environment variables
cp .env.example .env              # or create .env manually
# Add: GROQ_API_KEY=your_key_here
```

### Every time

```
# Start Milvus (vector database)
docker-compose up -d

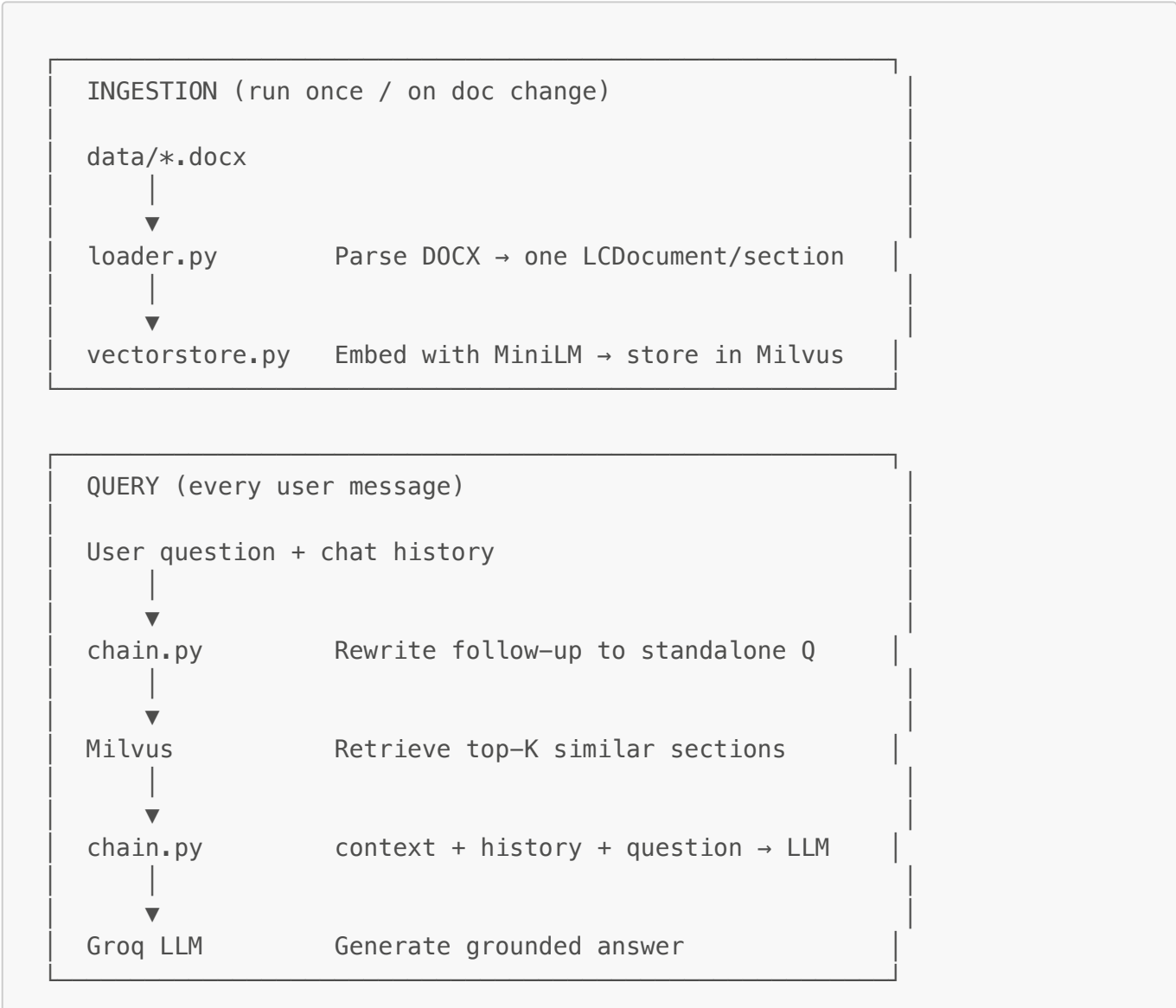
# Ingest documents (first time, or when source docs change)
python ingest.py

# Run the chatbot
python main.py
```

Useful URLs while running

| Service          | URL                   |
|------------------|-----------------------|
| Attu (Milvus UI) | http://localhost:3011 |
| Milvus gRPC      | localhost:19530       |
| MinIO console    | http://localhost:9001 |

4. Architecture Overview



## 5. Document Loading & Parsing

File: rag/loader.py

### The problem with standard DOCX loaders

Generic loaders split documents by page or fixed character count. For a financial product document this is wrong — a product's interest rate table and its requirements may split across chunks, losing the connection between them.

### Section-aware parsing strategy

The loader walks every element in the DOCX body **in document order** and groups content under heading boundaries. Each section becomes one `LCDocument`.

Heading1: "FIXED DEPOSIT ACCOUNT"

paragraph: "Minimum deposit: ..."

table: interest rate schedule

→ new document

→ appended to document

→ converted to text, appended

Heading2 starting with digit: "1001"

paragraph: "..."

table: charges

→ new document (product ID)

**Key rule:** `Heading2` is only treated as a section boundary if it starts with a digit (product codes like `1001`, `2003`). A `Heading2` like "Charges" is treated as body text within the current section.

### Table conversion

Tables are converted to plain text so the LLM can read them:

**2-column tables** (key-value pairs like Interest Rate | 7.5%) become:

Minimum Balance: Rs. 10,000  
Interest Rate: 7.5% per annum  
Tenure: 1 year

**Multi-column tables** (comparison grids) become pipe-separated rows:

Tenure | Regular Rate | Senior Citizen Rate  
1 Year | 7.50% | 8.00%  
2 Year | 7.75% | 8.25%

### Output format

Each `LCDocument` has:

```
LCDocument(  
    page_content="Full section text including tables as plain text",  
    metadata={  
        "source": "data/AI CHATBOT DEPOSITS.docx",  
        "section": "Griha Laxmi Scheme",    # heading text  
        "section_index": 4,                  # position in file  
    }  
)
```

Public functions

```
from rag.loader import load_docx_by_section, load_multiple_docx,  
find_docx_files  
  
# Parse single file → list of LCDocuments  
docs = load_docx_by_section("data/myfile.docx")  
  
# Parse multiple files at once  
docs = load_multiple_docx(["data/file1.docx", "data/file2.docx"])  
  
# Auto-discover all .docx in a directory  
paths = find_docx_files("data")
```

---

## 6. Embeddings

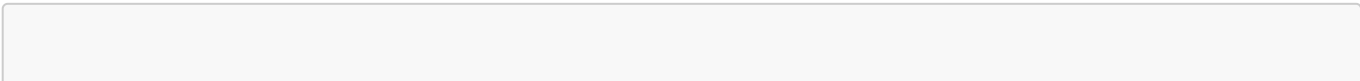
**File:** `rag/vectorstore.py` — `get_embeddings()`

**Model:** `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

| Property      | Value                                   |
|---------------|-----------------------------------------|
| Dimensions    | 384                                     |
| Languages     | 50+ (including Nepali, Hindi, English)  |
| Device        | CPU                                     |
| Normalization | L2 normalized (cosine similarity ready) |

The model is downloaded from HuggingFace on first use and cached locally in `~/.cache/huggingface/`. Subsequent runs load from cache — no internet needed after first run.

**Why this model:** BFC documents may contain mixed-language content. The multilingual MiniLM handles this while remaining lightweight enough for CPU inference.



```
from rag.vectorstore import get_embeddings

embeddings = get_embeddings()
vector = embeddings.embed_query("What is the interest rate?")
# → list of 384 floats
```

---

## 7. Vector Store (Milvus)

**File:** `rag/vectorstore.py`

Milvus stores document vectors and their associated text. At query time it performs approximate nearest-neighbor search to find the most relevant sections.

### Collection

- **Name:** `bfc_deposits` (set in `config.py`)
- **Schema:** managed automatically by `langchain-milvus`
- **Index:** auto-created on first `from_documents()` call

### Two operations

**Create** (ingestion — replaces all existing data):

```
from rag.vectorstore import create_vectorstore

vectorstore = create_vectorstore(documents)
# drop_old=True means previous data is wiped and replaced
```

**Connect** (query time — attaches to existing collection):

```
from rag.vectorstore import get_vectorstore

vectorstore = get_vectorstore()
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

### Retriever

```
retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})
# Returns top-K most similar LCDocuments for a given query string
# TOP_K = 5 by default (configured in config.py)
```

The retriever uses **cosine similarity** on the normalized vectors. Raising `TOP_K` gives more context to the LLM but increases token usage and may add noise.

---

## 8. Ingestion Pipeline

File: `ingest.py`

Full sequence when you run `python ingest.py`:

```
1. find_docx_files("data")
   └─ glob("data/*.docx") → ["data/AI CHATBOT DEPOSITS.docx"]

2. load_multiple_docx(paths)
   └─ For each file: load_docx_by_section(path)
      └─ Walk DOCX body elements
         ├── Heading1/Heading2(digit) → new section
         ├── Paragraph → append text to current section
         └── Table → convert to text, append to current section
      └─ Returns: List[LCDocument] (one per section)

3. create_vectorstore(documents)
   ├── get_embeddings() → load MiniLM model
   ├── Embed all page_content strings → 384-dim vectors
   └── Milvus.from_documents()
      ├── drop_old=True → wipe existing collection
      └── Store vectors + text + metadata in "bfc_deposits"
```

**Re-ingestion:** Any time source documents change, re-run `python ingest.py`. It always does a full replace (`drop_old=True`) — there is no incremental update.

---

## 9. LLM & Prompts

File: `rag/chain.py`

### LLM

- **Provider:** Groq
- **Model:** `meta-llama/llama-4-scout-17b-16e-instruct`
- **Temperature:** `0` — fully deterministic, no creativity
- **Auth:** `GROQ_API_KEY` from `.env`

### Two prompts

#### 1. Contextualization Prompt (`CONTEXTUALIZE_PROMPT`)

Used only when there is chat history. Rewrites follow-up questions into standalone ones before retrieval.

```
Given the chat history and the latest user question, rewrite the question
as a fully standalone question that can be understood without the chat
```

```
history.
Do NOT answer it – just rewrite if needed, otherwise return it as-is.
```

### Example:

```
History: Q: "What is Griha Laxmi Scheme?"
        A: "Griha Laxmi is a deposit scheme for..."
Follow-up: "What about interest rate in it?"
Rewritten: "What is the interest rate for Griha Laxmi Scheme?"
```

The rewritten question is used for Milvus retrieval. The original question is still used in the final answer prompt.

## 2. QA System Prompt (**SYSTEM\_PROMPT**)

Governs the final answer generation. Key rules enforced:

1. Answer **only** from provided context — no outside knowledge
2. If not in context → return a fixed "not found" message
3. Reproduce numbers/rates/requirements **exactly** — no paraphrasing
4. List all matching products if multiple match
5. Strict no-guessing, no inference

The `{context}` (retrieved document text) is injected at the bottom of the system prompt.

---

## 10. RAG Chain

**File:** `rag/chain.py` — `get_rag_chain(retriever)`

Built entirely from `langchain_core` LCEL (no `langchain.chains` dependency).

Chain flow

```
def invoke(inp: dict) -> dict:
    # inp = {"input": str, "chat_history": list[BaseMessage]}

    # Step 1 – Rewrite question if follow-up
    if inp["chat_history"]:
        standalone = (contextualize_prompt | llm |
StrOutputParser()).invoke(inp)
    else:
        standalone = inp["input"]

    # Step 2 – Retrieve relevant sections from Milvus
    docs = retriever.invoke(standalone)
    context = "\n\n---\n\n".join(doc.page_content for doc in docs)
```



```
# Step 3 – Generate answer
answer = (qa_prompt | llm | StrOutputParser()).invoke({
    "input": inp["input"],          # original question (not
    rewritten)
    "chat_history": inp["chat_history"],
    "context": context,
})

return {"answer": answer}
```

## Usage

```
from rag.chain import get_rag_chain

chain = get_rag_chain(retriever)
result = chain.invoke({"input": "What is Griha Laxmi?", "chat_history":
[]})
print(result["answer"])
```

## 11. Multi-Turn Conversation

**File:** `main.py`

Chat history is maintained as a list of `HumanMessage` / `AIMessage` objects from `langchain_core.messages`. It lives in memory for the session — cleared on restart.

```
chat_history = []

# Each turn:
result = rag_chain.invoke({"input": question, "chat_history":
chat_history})
answer = result["answer"]

chat_history.extend([
    HumanMessage(content=question),
    AIMessage(content=answer),
])
```

The history is passed to both prompts:

- **Contextualization prompt** — to rewrite follow-ups before retrieval
- **QA prompt** — so the LLM can refer to prior answers naturally

**Limitation:** History is not persisted to disk. If persistence across sessions is needed, serialize `chat_history` to a file or database and reload on startup.

## 12. Configuration Reference

File: `rag/config.py`

| Variable                     | Default                                                | Description                            |
|------------------------------|--------------------------------------------------------|----------------------------------------|
| <code>GROQ_API_KEY</code>    | from <code>.env</code>                                 | Groq API credentials                   |
| <code>MILVUS_HOST</code>     | <code>localhost</code>                                 | Milvus server host                     |
| <code>MILVUS_PORT</code>     | <code>19530</code>                                     | Milvus gRPC port                       |
| <code>COLLECTION_NAME</code> | <code>bfc_deposits</code>                              | Milvus collection name                 |
| <code>EMBEDDING_MODEL</code> | <code>paraphrase-multilingual-MiniLM-L12-v2</code>     | HuggingFace embedding model            |
| <code>LLM_MODEL</code>       | <code>meta-llama/llama-4-scout-17b-16e-instruct</code> | Groq model ID                          |
| <code>TOP_K</code>           | <code>5</code>                                         | Number of sections retrieved per query |

`.env` file

```
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXX
USER_AGENT=bfc-deposit-assistant
```

## 13. Infrastructure (Docker)

File: `docker-compose.yml`

Four services, all on the `milvus` Docker network:

etcd (port 2379)  
Distributed key-value store.  
Milvus uses it for service discovery & cluster metadata.

MinIO (port 9000 / console 9001)  
S3-compatible object storage.  
Milvus stores segment files here.  
Credentials: minioadmin / minioadmin

Milvus Standalone (port 19530 / 9091)

```
The vector database.  
Depends on etcd + MinIO being healthy.  
Health check: GET /healthz
```

```
Attu (port 3011)  
Web UI for browsing Milvus collections,  
running queries, inspecting vectors.  
Opens only after Milvus is healthy.
```

**Persistent volumes** are stored in `./volumes/` (gitignored). Deleting this folder wipes all vector data — you'll need to re-run `python ingest.py`.

```
docker-compose up -d      # start all services  
docker-compose down       # stop (data preserved in volumes/)  
docker-compose down -v    # stop AND delete volumes (full reset)
```

---

## 14. Adding New Documents

1. Drop any `.docx` file into the `data/` directory.
2. Re-run ingestion:

```
python ingest.py
```

3. That's it — the loader auto-discovers all `.docx` files in `data/`.

### Notes:

- `create_vectorstore()` always does a full replace (`drop_old=True`). All documents in `data/` are re-ingested together every time.
- The document must use Word heading styles (`Heading 1`, `Heading 2`) for section detection to work. Plain bold text is not detected as a heading.
- Tables must be actual Word tables, not text formatted to look like tables.

### Changing the LLM

Update `LLM_MODEL` in `rag/config.py`. Any Groq-supported model ID works. See Groq's model list for options.

### Changing the embedding model

Update `EMBEDDING_MODEL` in `rag/config.py`, then **re-ingest** — stored vectors must match the new model's dimensions. Using a different model without re-ingesting will produce garbage retrieval results.

## 15. Common Issues

### No module named 'langchain.chains'

`langchain` v1.x removed the `chains` module. The codebase uses only `langchain_core` — do not import from `langchain.chains`.

### ModuleNotFoundError on any `rag.*` import

Run scripts from the project root (`rag_best/`), not from inside the `rag/` folder. The `rag` package must be on the Python path.

```
# Correct
cd rag_best && python main.py

# Wrong
cd rag_best/rag && python ../main.py
```

### Milvus connection refused

Milvus takes ~30–90 seconds to become healthy after `docker-compose up`. Wait for the health check to pass before running `ingest.py` or `main.py`.

```
docker-compose ps    # check status – should show "healthy"
```

### Retrieval returns wrong sections

- The embedding model may not be cached yet — first run downloads ~500MB.
- If you changed `EMBEDDING_MODEL`, re-run `python ingest.py`.
- Try increasing `TOP_K` in `config.py` to retrieve more candidates.

### LLM returns "I couldn't find this information"

The answer genuinely isn't in the source document, or the relevant section wasn't retrieved. Use Attu (<http://localhost:3011>) to inspect what's stored in the `bfc_deposits` collection and verify the section exists.